

DEEP LEARNING

Labwork 2

Students Ho Thanh Thuy Tien - 2540100

1 Implementation

Model and Loss Function

First of all, I implemented a simple linear regression model where the relationship between the independent variable x and the dependent variable y is represented by the linear equation:

$$y = w_1x + w_0$$

To measure the accuracy of our model, I used the Mean Squared Error as our loss function:

$$J = \frac{1}{N} \sum_{i=1}^N L_i = \frac{1}{2} * \frac{1}{N} * \sum_{i=1}^N (w_1 * x_i + w_0 - y_i)^2$$

1.1 Gradient Descent Update Rules

I calculated the gradients for w_1 and w_0 , then updated the weights by subtracting them to minimize the loss.

- Partial Derivative for w_1 :

$$\frac{\partial J}{\partial w_1} = \frac{1}{N} \sum_{i=1}^n (w_1x_i + w_0 - y_i) \cdot x_i$$

- Partial Derivative for w_0 :

$$\frac{\partial J}{\partial w_0} = \frac{1}{n} \sum_{i=1}^n (w_1x_i + w_0 - y_i)$$

The weights are updated iteratively using a learning rate r (w_1 also have the same function):

$$w_0 = w_0 - r * \frac{dL}{dw_0}$$

And the gradient stop iterating when the change in the loss between two consecutive steps fell below a specific threshold (which is $1e-6$).

2 Learning Rate Analysis

I tested the model with various learning rates (r).

- **Low Learning Rate** ($r = 0.0001$): It took 179,621 iterations to reach the threshold. While the result is highly accurate, the computational cost is significant because the steps taken along the gradient are very small.
- **Optimal Learning Rate** ($r = 0.0006$): It reached a similar loss in only 35,445 iterations. This is approximately 5 times faster than the lower learning rate
- **Excessive Learning Rate** ($r = 0.002$): At this r , loss grows exponentially because excessive step sizes cause the model to bypass the minimum, leading to divergence and numerical failure.

3 Visualization

As you can see, the curve fit all data points

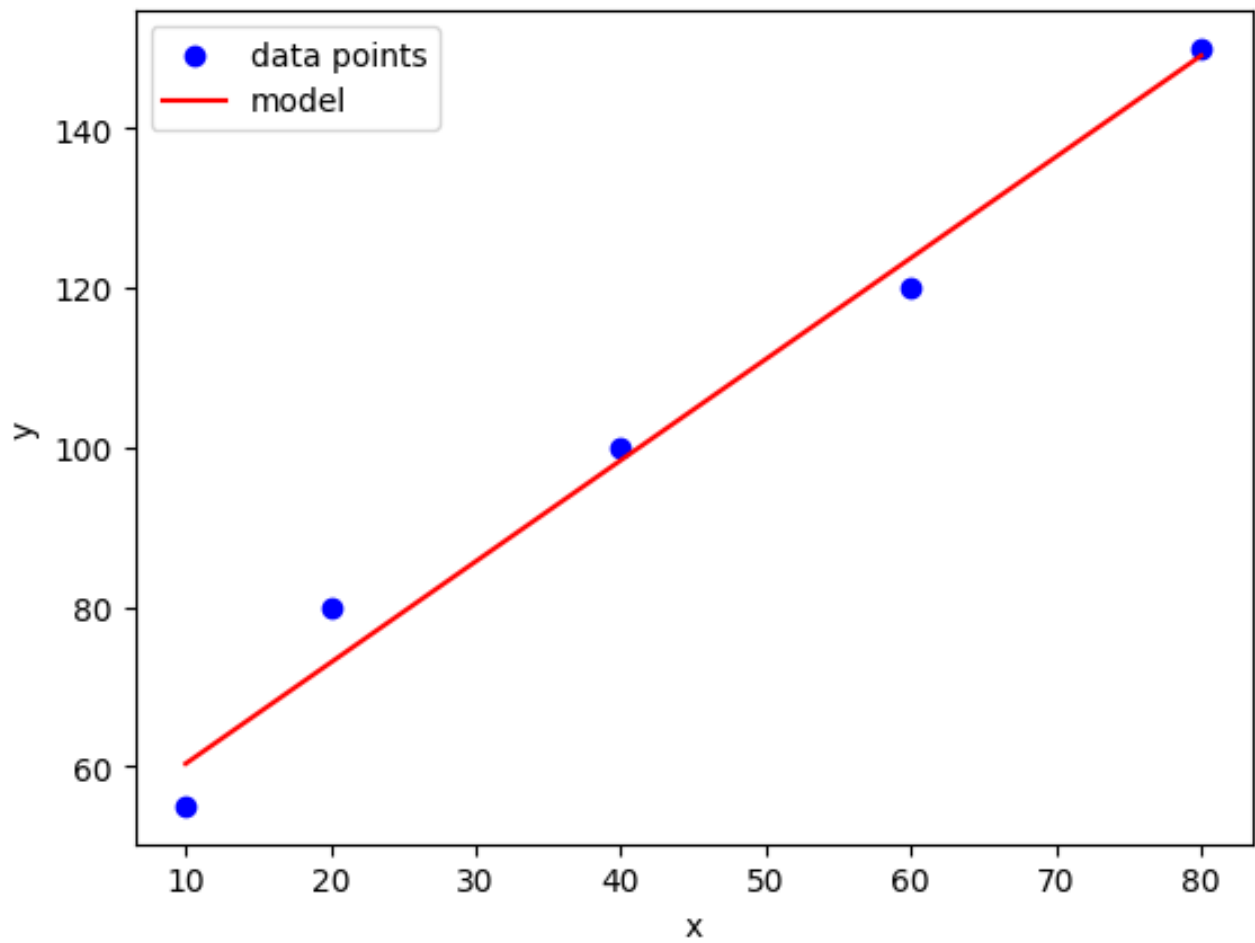


Figure 1: Linear regression curve